TR-2026-06

Capability Measures the Driving, Not the Engine

A three-tier result dissolved, and a defect found in the author's own engine

Noah Parsons

ORCID 0009-0000-7224-6040

5 September 2026

Abstract

The frozen scaling sweep reported that three rigid-body dynamics engines answer the planar N-link chain up to 5, 12 and 30 links respectively, and this was read as a three-tier capability separation. It is not one. Driving each engine along a different path through the same public interface, and adjudicating every result against the same closed-form reference, all three reach 30 — the top of the ladder. The reported tiers measured three different driving styles. Pursuing MechanicsDSL further found two further limits, neither of them inherent. 97% of compilation was spent in one `sympy.simplify` call whose result is cosmetic; removing it made the engine 37× faster at $N = 8$. What then blocked it at $N = 32$ was CPython's default recursion limit — an interpreter default, not a property of the problem. Raised, the engine derives a **100-coordinate system** in 4414 s, agreeing with the reference to 1.3×10^{-12}. Pushing only one engine would have introduced a fresh asymmetry favouring the author's own, so all three were then asked for the same systems under the same clock in one session: **all three reach** 80 **coordinates**, and reach is not a differentiator between them at all. The engine already intended to skip the costly call under a deadline and could not, because on Windows that deadline was inert and its expiry was raised on a thread where no caller could catch it. Three defects are fixed here. **Every measurement in this report agreed with the closed-form reference**, so nothing here bears on correctness — only on reach, and on controls that reported working while doing nothing.

1 Why this was run

TR-2026-04 reported a scaling ladder in which `sympy.physics.mechanics` answered chains up to 5 links, MechanicsDSL up to 12, and Drake up to 30, each under a 120 s wall clock. The natural reading — three tiers of capability — was carried into the paper's abstract.

That reading has a weakness which a reviewer would find before a reader does. Each engine was driven along exactly one path, and for SymPy that path was the one its documentation presents: build a `LagrangesMethod` and call `rhs()`. Internally `rhs()` evaluates `mass_matrix_full.LUsolve(forcing_full)`, a *symbolic* linear solve whose cost grows explosively with the number of coordinates. A wall attributable to one interior operation is a fact about that operation, not about the library.

The study's own adapter anticipated this. Its docstring records that a second mode exists and that "the gap between them is a measure of how much the documented path costs a user relative to the informed one." That secondary result was never measured. This report measures it, and then — because measuring it for one contestant and not the others would be an asymmetry favouring the author's own engine — repeats the exercise for MechanicsDSL.

2 Method

Each engine is driven along several paths through its own public interface, on the same N-link chain, under the same wall clock, and adjudicated against the same closed-form reference of `reference.py`. Nothing frozen is altered: no case is removed, no recorded figure is restated, and the engine under test remains pinned at `a8dc2b2`. This is harness growth, which the freeze permits.

Correctness gates every timing. A faster wrong answer is worth nothing, and three of the four faults recorded in this study's own apparatus (TR-2026-04, Table 1) were convention mismatches that produced clean constant error ratios. Every variant is therefore validated against the reference at 16 probe states before its build time is allowed to count. A variant that is fast and disagrees is reported as DISAGREE, not as a success. **No variant disagreed.**

A timing caveat. The machine used for this report ran roughly twice as slowly as the one that produced the frozen sweep: the unmodified MechanicsDSL path reached 8 links here against 12 in the record. Every comparison below is therefore same-session, variant against variant. The archived figures are not restated from this run and remain as recorded.

3 SymPy: the wall was the symbolic solve

Four ways of driving `sympy.physics.mechanics`, all public API:

Variant	Formulation	Solve
`idiomatic`	`LagrangesMethod` → `rhs()`	symbolic (the frozen path)
`lagrange_cse`	`LagrangesMethod`, M and F kept	numeric, `lambdify(cse=True)`
`kane`	`KanesMethod` → `rhs()`	symbolic
`kane_cse`	`KanesMethod`, M and F kept	numeric, `lambdify(cse=True)`

N	`idiomatic`	`lagrange_cse`	`kane`	`kane_cse`
3	1.61 s	1.35 s	1.51 s	1.40 s
5	5.16 s	2.45 s	4.57 s	1.52 s
6	20.08 s	5.32 s	33.56 s	1.82 s
7	TIMEOUT	10.30 s	TIMEOUT	1.97 s
8	—	25.34 s	—	2.09 s
10	—	TIMEOUT	—	1.81 s
12	—	—	—	4.41 s
20	—	—	—	18.62 s
30	—	—	—	64.99 s
40	—	—	—	TIMEOUT

Neither remedy suffices alone, and this is the informative part. Kane's method by itself buys nothing — 6 links, the same as idiomatic, and slower at $N = 6$. Deferring the solve and applying CSE buys two links, 6 to 8. Together they reach 30. The two compound rather than add, because the numeric solve is what removes the `LUsolve` blow-up, and only once that is gone does Kane's cheaper mass matrix govern the cost.

Driven as its documentation presents it, SymPy reaches 6. Driven by a user who knows the library, it reaches 30.

4 MechanicsDSL: the wall was one cosmetic call

4.1 Two hypotheses, both wrong

MechanicsDSL was never subject to SymPy's disease. At `NUMERIC_MASS_MATRIX_MIN_COORDS = 4` (`compiler.py:69`) it already returns `{"__mass_matrix__": (M, F)}` and defers the M^{-1} solve to evaluation time. The remedy that carried SymPy from 8 to 30 was in place here from the start.

The remaining candidate was CSE: `solver/core.py` lambdified M and F without it, and the chain's mass matrix has $O(N^2)$ entries built from cosines of angle differences. It made matters worse — 107 s against 86.5 s at $N = 8$ — because CSE's own analysis costs time and lambdify was not where the time was.

4.2 What the profile showed

Profiling the $N = 8$ compile settled it:

Location	Share	Nature
`sympy.simplify`	97%	cosmetic rewriting
→ `trigsimp._futrig`	—	
→ `polytools.factor`	—	multivariate polynomial factorisation
→ `densebasic.py` (self time)	48%	
→ `densearith.py` (self time)	24%	

Simplification rewrites an expression into an equivalent one. It changes the *size* of the derived equations and never their *value*. The engine was spending almost all of its compile budget on legibility.

4.3 The measurement

Variant	$N = 8$ **build**	Reach	Relative error
`frozen`	86.5 s	8	$2.1663226767998367 \times 10^{-14}$
`cse`	107.0 s	8	$2.1663226767998367 \times 10^{-14}$
`fastsimp`	88.5 s	8	$2.1663226767998367 \times 10^{-14}$
`nosimplify`	**2.04 s**	**30**	$2.1663226767998367 \times 10^{-14}$

The error is identical to the last digit across all four. At $N = 30$ the compile takes 112 s. Past that the engine *refuses*, raising `RecursionError` and returning `success: False` — a loud failure, not a wrong answer, and the correct behaviour when it cannot proceed. But the reason it could not proceed turned out not to be its own.

4.4 The second limit was CPython's, not the engine's

Deriving a strongly coupled system walks expression trees whose depth grows with the number of coordinates, and past roughly thirty the planar chain exceeds CPython's default recursion limit of 1000. That default, not the mathematics and not the engine, is what refused $N = 32$.

Raising the limit — necessarily together with the stack, since a raised limit on a default stack converts a catchable `RecursionError` into a hard interpreter crash — changes an instant refusal into a completed derivation:

N	Build	Relative error	Note
30	112 s	9.76×10^{-14}	default limit suffices
32	145 s	9.13×10^{-14}	*refuses* at the default limit
36	181 s	1.19×10^{-13}	
40	279 s	1.64×10^{-13}	
48	467 s	2.38×10^{-13}	
60	916 s	4.09×10^{-13}	
80	2338 s	5.95×10^{-13}	
100	4414 s	1.28×10^{-12}	measurement stopped here

Cost grows as roughly $O(N^3)$: doubling 30 to 60 costs 8.2× the time, and the $N = 100$ figure of 4414 s came within two per cent of the extrapolation made from $N = 60$. The residual grows far more slowly, from 9.8×10^{-14} at $N = 30$ to 1.3×10^{-12} at $N = 100$, which is the expected accumulation as the mass matrix's condition number rises with N and remains four orders of magnitude inside the 10^{-8} adjudication tolerance.

The engine derives a 100-coordinate system correctly. *No ceiling was found.* The ladder was stopped at 100 because the next rung would have taken about two hours, not because the engine failed or slowed abnormally. The eventual limit is expected to be memory rather than time — 10^4 mass-matrix entries are held as expression trees simultaneously — but that is a prediction, not a measurement.

5 A control that reported working while doing nothing

The engine already intended to skip this cost. Seven call sites wrapped simplification in a deadline and, on expiry, logged "using unsimplified equation" and continued. The intent was never delivered, for two independent reasons.

First: the deadline could not interrupt the work. `config.simplification_timeout` was implemented with a watchdog thread. A Python thread cannot interrupt SymPy inside a C-level routine, which is exactly where expensive simplification spends its time. Setting the deadline to 0.05 s made the expiry message appear and left the compile at 88.5 s.

Second: the expiry was raised where nobody could catch it. On Windows the handler executed `raise TimeoutError(...)` *inside the timer thread.* An exception raised there cannot propagate to the caller: it printed a traceback to `stderr`, killed the timer thread, and left every `except TimeoutError` around the call sites permanently unreachable. A `threading.Event` was set and never read. The deadline was inert and noisy at once.

Third: the off switch selected the slowest path. The guard read `if config.simplification_timeout > 0`, with an `else` branch calling `sp.simplify` unguarded. Setting the value to 0 — the value a user reaches for to turn a timeout off — did not disable simplification. It removed the deadline and let simplification run unbounded.

Fourth: an interpreter default presented as a derivation failure. The `RecursionError` surfaced as "Equation derivation failed: maximum recursion depth exceeded", which reads as the engine finding the system intractable. It was CPython's default limit. A user meeting that message has no indication that the system is well within reach and that one setting would admit it.

This is not an instance of the study's central claim: no wrong number was returned, and the engine's physics was never affected. It is the same shape one level down — a control that reports success and has no effect — and it was found in the author's own engine by the same discipline of consulting an artefact rather than reasoning further.

6 The fix

1. **A real switch.** `config.enable_simplification` (default `True`, preserving existing behaviour) skips simplification outright. This is the only reliable way to avoid the cost.
2. **One policy, one place.** All seven call sites now route through `utils/simplification.py:maybe_simplify()`, which owns the decision, never propagates a simplification failure, and names the quantity in its warnings.
3. **An honest deadline.** The Windows timer now sets its flag in the thread and raises on the *caller's* thread after the body returns. The expiry is catchable and the `stderr` tracebacks are gone. It remains advisory, and both the docstring and the warning now say so.
4. **The trap removed.** Passing 0 warns that it removes the deadline rather than the simplification, and points at `enable_simplification`.
5. **Deep recursion.** A `RecursionError` in derivation now retries once with a raised limit on an enlarged stack (`utils/recursion.py`), governed by `config.deep_recursion` (default on), `recursion_limit` and `derivation_stack_mb`. Only the derivation is retried, not the whole compile, so parsing is not repeated. If every stack size is refused the retry runs inline rather than failing. The residual error message now states that the limit was the interpreter's and names the settings.
6. **A latent defect fixed on the way.** Running derivation on a worker thread would have broken the Unix deadline outright: `signal.alarm` is main-thread only and raises `ValueError` elsewhere. `timeout()` now detects that case and degrades to the advisory behaviour instead of crashing.

Measured through the supported switch, on one machine in one session: $N = 5$, $14.15 \to 1.10\,\text{s}$ (12.9×); $N = 8$, $112.8 \to 3.05\,\text{s}$ (37.0×). Accelerations bit-identical in both.

This fix is not in the study's engine. It lives on `fix/simplification-cost`. `FREEZE.md` is explicit that if the engine moves off `a8dc2b2` the baseline is void, and the frozen figures were measured on the unfixed engine. The study's pin stands, and the `v2.1.3` tag keeps it recoverable.

7 The same question, the same budget, to all three

Establishing that MechanicsDSL reaches 100 while leaving SymPy and Drake at the 30 where the frozen ladder happened to stop would have replaced one asymmetry with another, and this one would favour the author's own engine. Neither of the other two failed at 30; neither was asked for more.

Every engine was therefore asked for the same systems, each along the best path found for it, under the same wall clock, in one process on one machine, adjudicated against the same reference. Drake has no native Windows build, so the comparison was made in WSL where all three import together — otherwise engine and operating system would be confounded.

	Drake		**SymPy**		**MechanicsDSL**	
N	time	residual	time	residual	time	residual
40	0.3 s	1.7×10^{-10}	74.3 s	3.6×10^{-13}	145.0 s	1.7×10^{-13}
60	0.2 s	4.1×10^{-10}	280.2 s	7.1×10^{-13}	468.0 s	4.2×10^{-13}
80	0.2 s	9.9×10^{-10}	693.3 s	1.2×10^{-12}	1100.9 s	5.9×10^{-13}

All three reach 80 **coordinates. No engine failed at any rung, and no engine disagreed with the reference.** Reach, which the frozen sweep reported as a three-tier separation, is not a differentiator between these engines at all.

Three things in this table are worth stating precisely, and two of them are unfavourable to the author's engine.

- **Drake is in a different class.** Its cost is flat at 0.2–0.3 s across the whole ladder — an $O(N)$ articulated-body algorithm against two symbolic approaches growing as roughly $O(N^3)$. At $N = 80$ that is a factor of some 5500. Its residual is larger (10^{-9} against 10^{-13}) but remains far inside tolerance.
- **Driven well, SymPy is faster than MechanicsDSL at every rung** — 74 against 145 s, 280 against 468 s, 693 against 1101 s, consistently about 1.6×. Of the two symbolic engines the author's is the slower.
- **MechanicsDSL returns the smaller residuals on this family**, by roughly a factor of two at every rung, and **that advantage does not survive a change of mechanism.** On the slider–crank SymPy leads (median 4.8×10^{-16} against 7.5×10^{-16}); on the cart–pole MechanicsDSL has the worst case of the three (1.0×10^{-12}). The lead is a property of the planar chain rather than of the formulation and should not be claimed generally.

  A caution attaches to the cart–pole column, and it is an instance of the diagnostic of §6: SymPy's residual there is *exactly* zero across 144 probes. That is not accuracy. SymPy derives the identical mass matrix $\left[\begin{smallmatrix} M+m & ml\cos\theta \\ ml\cos\theta & ml^2 \end{smallmatrix}\right]$ to the hand reference and then performs the same `numpy` solve on the same inputs, so the arithmetic is bit-identical and the residual measures nothing. The derivations agreeing exactly is a real and strong result; the zero cannot rank the engines, and **no accuracy ordering should be read from that column at all.**

Provenance. Every row above comes from one engine state in one session on one platform. The MechanicsDSL ladder of §4.4 does not: its rows were taken across three code states as the fixes here were developed, and WSL runs roughly twice as fast as Windows for this work ($N = 80$: 1101 s here against 2338 s there). The two tables must not be mixed. Where they differ, this one governs.

8 Consequences for the study

Engine	As driven (frozen)	Equal budget	Time at $N = 80$
`sympy.physics.mechanics`	5	**80**	693 s
MechanicsDSL	12	**80**	1101 s
Drake	30	**80**	**0.2 s**

The three-tier capability claim does not survive, and nothing replaces it. Given the same systems and the same clock, all three engines answer every rung to 80 coordinates. The tiers reported in the frozen sweep measured how each engine happened to be driven, and 80 is where the ladder stopped rather than where any engine failed.

What separates the engines is speed, and there the ordering is unambiguous. Drake's $O(N)$ recursive algorithm is flat across the ladder and beats both symbolic engines by some 5500× at $N = 80$; that is a difference of algorithm class rather than of engineering quality, and no change of driving affects it. Between the two symbolic engines, SymPy driven well is about 1.6× faster than MechanicsDSL at every rung. MechanicsDSL returns residuals about twice as small *on this family*, an advantage that reverses on the cart–pole and the slider–crank and so should not be generalised. **On the axis the frozen sweep was read as separating them, they do not separate.**

Two asymmetries must be reported together, because they point in opposite directions and quoting either alone would mislead.

- **Against MechanicsDSL.** SymPy's 30 is reachable by any competent user reading the documentation. MechanicsDSL's 30 required a fix to the engine; before it, no documented means existed, because the one control provided did not work.
- **For MechanicsDSL.** Its ceiling was never architectural. It already avoided the symbolic solve that limited SymPy, and neither limit it did have was inherent: one accidental cost in a cosmetic call, and one interpreter default. Both are now removed, and the engine derives systems more than three times the size of anything the frozen sweep asked of any engine.

One default still stands in the way. The 100-coordinate result requires `enable_simplification = False`, which is off by default. Out of the box a user still meets the simplification cost long before the recursion limit. That default was deliberately left alone: changing it would alter the derived equations of every existing system from tidy to bulky, which is a visible behavioural change and a release decision rather than a bug fix. Disabling simplification automatically above some coordinate count is the obvious follow-up.

What the paper must say instead. The abstract's "chains of 5, 12 and 30 links respectively" and §1.6's "factor-of-six capability difference" both state as engine properties what are properties of the driving. The defensible claim is narrower and more interesting: *reach depends on the path taken through the tool, the documented path was the slowest of the four measured for SymPy, and the capability tiers reported in the frozen sweep dissolve when each engine is driven by an informed user.*

What is unchanged. Every correctness finding stands. Across 50 measurements in this report — four SymPy variants and five MechanicsDSL variants over two ladders — there were **zero disagreements with the closed-form reference**. No engine returned a wrong answer while reporting success, under any driving style, at any N. Claim (A) of the study is untouched and claim (B) remains unsupported.

9 Reproduction

```
cd stress_suite
python sweep_sympy_variants.py --wall 120
python sweep_mdsl_variants.py  --wall 150
python profile_mdsl.py 8
```

Records: `out/sympy_variants.json`, `out/mdsl_variants.json`. The engine fix is on `fix/simplification-cost`; the study's engine remains `a8dc2b2`.